BABU BANARASI DAS UNIVERSITY

LUCKNOW

SCHOOL OF COMPUTER APPLICATION

Big Data Fundamentals Project

BCADS-22

Submitted by -

Group - A

Anvee Verma (1240258114)

Anmol Pandey (1240258096)

Ananya Shukla (1240258081)

Ayati Sharma (1240258138)

Submitted to -

Mr. Vikas

PROJECT

SOCIAL MEDIA ANALYSIS

❖ ARCHITECTURE

The architecture of this project follows a standard Big Data pipeline using Hadoop ecosystem components.

❖ DATA FLOW

Raw Dataset → HDFS → MapReduce Processing → Processed Data → Hive Analysis → Insights

❖ EXPLANATION

The raw dataset (social media data) is first stored in HDFS (Hadoop Distributed File System).

- MapReduce is used to process the data and classify sentiments (Positive/Negative).
- It also performs aggregation operations like counting sentiments.
- The processed output is stored back in HDFS.
- Hive is used to perform analytical queries on the processed data.
- Final insights such as sentiment distribution are generated from Hive results.

❖ TOOLS USED

- Hadoop Distributed File System (HDFS)
- MapReduce (Hadoop Streaming)
- Hive (for data analysis)
- Linux Terminal
- ChatGPT (Generative AI tool)

❖ TECHNOLOGIES

- Python (for Mapper and Reducer scripts)
- CSV Dataset
- Hadoop Ecosystem
- Cloudera Environment

❖ EXAMPLE PROMPT

"Write a Python MapReduce program to classify social media comments into positive and negative sentiment."

❖ VALIDATION

- All AI-generated code was manually tested before execution.
- Local testing was performed using Linux commands (cat, sort, python).
- Output was verified at each stage:
  - Mapper output
  - Reducer output
  - Hadoop job output
- Hive query results were cross-checked with expected results.

❖ DATASET DESCRIPTION

The dataset used in this project is a **social media dataset** containing user posts, engagement metrics, and textual comments.
It is stored in **CSV (Comma Separated Values)** format and used for sentiment and engagement analysis using Hadoop.

❖ DATASET STRUCTURE

The dataset contains the following columns:

- **id** – Unique identifier for each post
- **user** – Name of the user
- **platform** – Social media platform (Instagram, Twitter, etc.)
- **post_type** – Type of post (Reel, Tweet, Video, etc.)
- **likes** – Number of likes

- **shares** – Number of shares
- **comments** – Number of comments
- **comment_text** – Text content of user comment

❖ DATASET SOURCE

- The dataset is a **combination of simulated and real-world inspired data**.
- It is created for educational purposes to demonstrate Big Data processing using Hadoop.

❖ DATASET PURPOSE

The dataset is used to:

- Perform sentiment analysis (positive vs negative comments)
- Analyse user engagement across platforms
- Apply MapReduce for data transformation and aggregation
- Generate insights using Hive queries

❖ SAMPLE DATA

```
id,user,platform,post_type,likes,shares,comments,comment_text
1,Aman,Instagram,Reel,120,30,15,good amazing content
2,Riya,Twitter,Tweet,45,10,5,bad experience poor service
3,Rahul,Facebook,Post,200,50,25,awesome loved it
4,Priya,Instagram,Story,80,20,10,not good waste time
5,Amit,YouTube,Video,500,100,60,excellent content very nice
6,Sneha,Twitter,Tweet,30,5,3,very bad experience
7,Karan,Facebook,Post,150,40,20,amazing and good
8,Neha,YouTube,Video,400,90,55,poor quality bad content
9,Arjun,Instagram,Reel,220,60,30,loved it awesome
10,Meena,Twitter,Tweet,70,15,8,good but slow
11,Rohit,Facebook,Post,180,45,22,excellent and amazing
12,Pooja,Instagram,Reel,140,35,18,good nice post
13,Vikas,Twitter,Tweet,60,12,6,bad service
14,Anjali,YouTube,Video,450,95,50,awesome video loved it
15,Deepak,Facebook,Post,160,38,19,good and amazing
16,Simran,Instagram,Story,90,25,12,waste of time bad
17,Kunal,Twitter,Tweet,55,11,7,poor experience
18,Tina,YouTube,Video,380,85,48,excellent quality
19,Varun,Facebook,Post,210,55,28,loved it good
20,Nikita,Instagram,Reel,170,45,24,amazing content
```

❖ CONCLUSION

- In this project, a complete Big Data pipeline was successfully implemented using Hadoop ecosystem components.
- The dataset was stored in HDFS, processed using MapReduce to classify sentiments, and analysed using Hive to extract meaningful insights.
- The project provided a clear understanding of distributed data processing and large-scale data analytics.
- Generative AI played an important role in assisting with code generation, debugging, and improving understanding of concepts.
- Overall, the project demonstrates how Big Data tools can be used effectively to analyse real-world social media data.

❖ **Generative AI** was used not only for code generation but also as a learning assistant to understand Big Data workflows and improve overall efficiency

QUESTION-1:- SOCIAL MEDIA ANALYSIS USING HADOOP STREAMING.

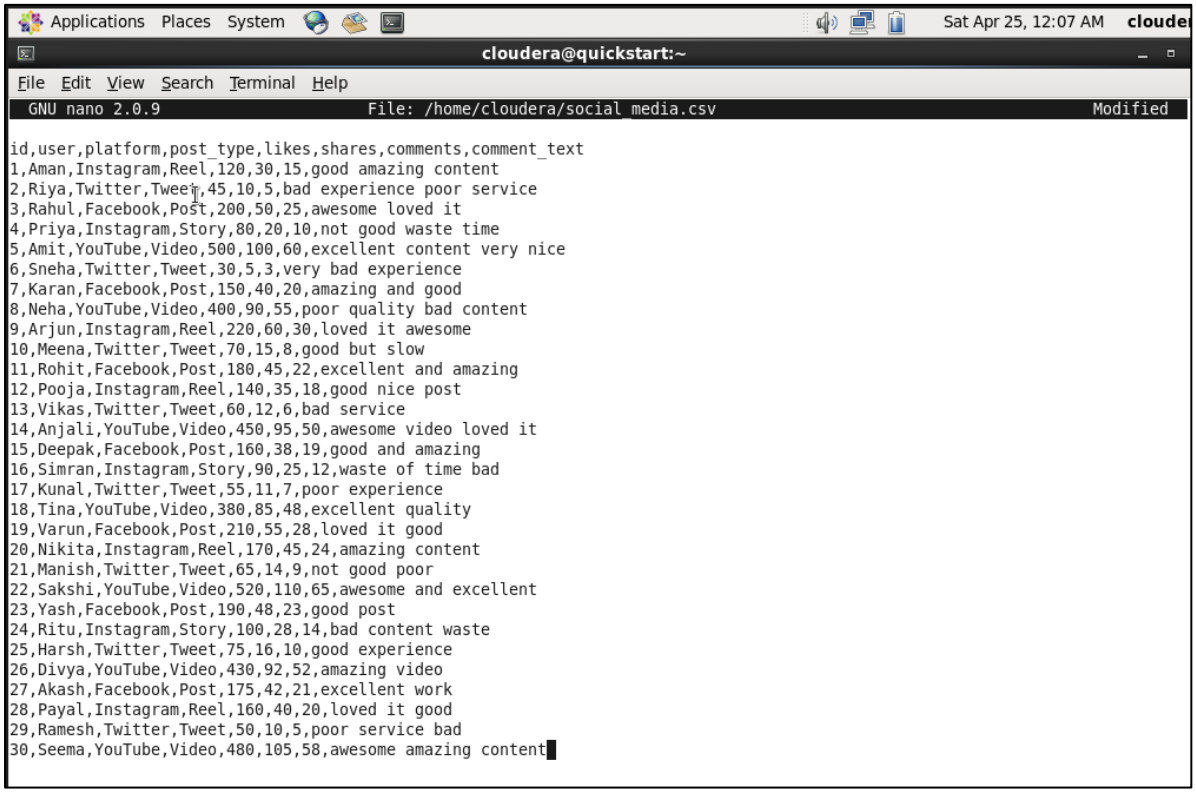
❖ AIM:

"TO ANALYZE SOCIAL MEDIA DATASET AND PERFORM SENTIMENT AND ENGAGEMENT ANALYSIS USING HADOOP MAPREDUCE AND HIVE."

Step-01: - Create a CSV file containing social media data including user details, platform, engagement metrics, and comment text.

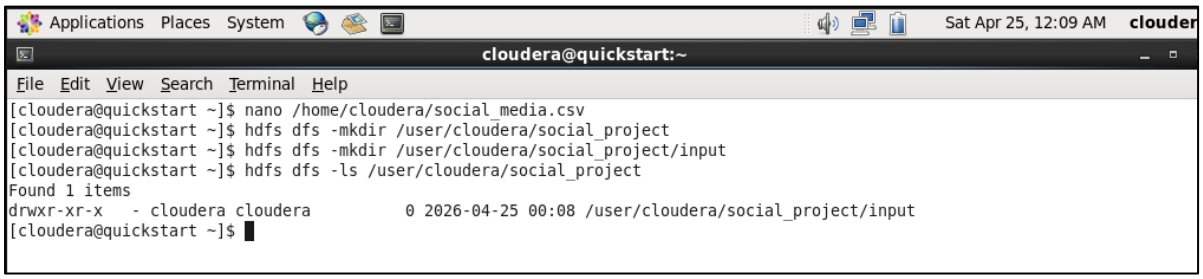
- nano /home/cloudera/social.csv

- Paste your dataset and save.



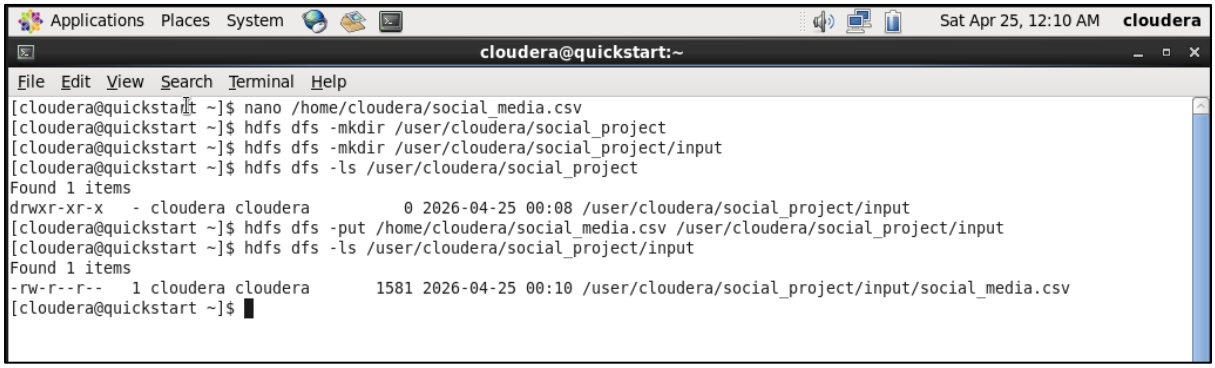
Step-02 : Create required directories in Hadoop Distributed File System (HDFS) to store input and output data.

- hdfs dfs -mkdir /user/cloudera/social_project
- hdfs dfs -mkdir /user/cloudera/social_project/input



Step-03 : Upload the CSV dataset from the local system to HDFS input directory and verify the upload.

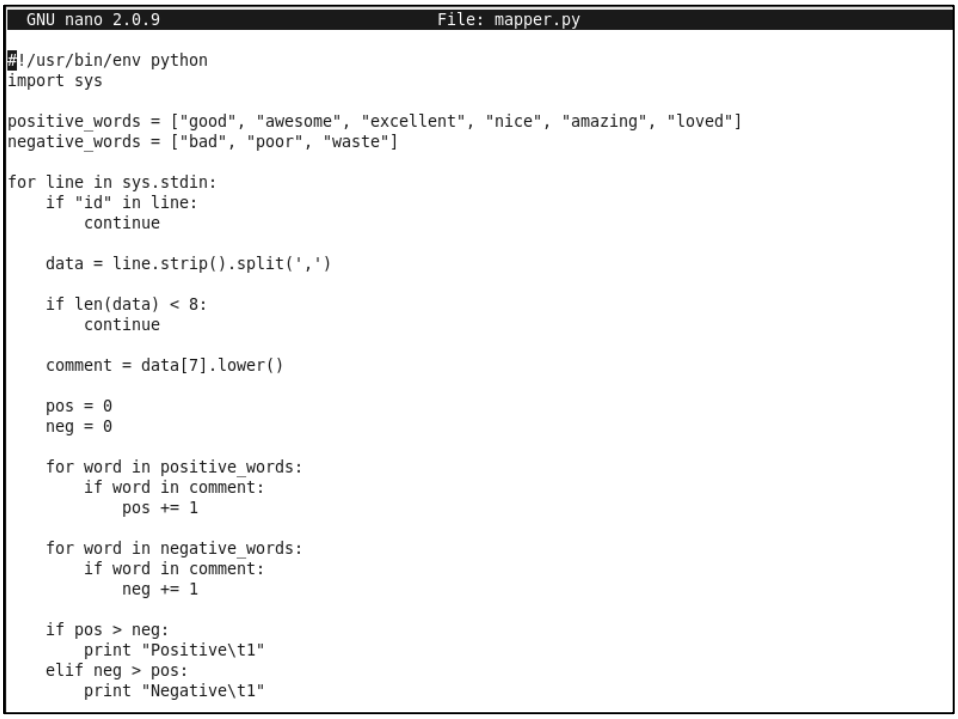
- hdfs dfs -put /home/cloudera/social_media.csv /user/cloudera/social_project/input
- hdfs dfs -ls /user/cloudera/social_project/input



Step-04 : Write a Python mapper script to process input data and classify comments into positive and negative sentiment**.**

- nano mapper.py

```
#!/usr/bin/env python
import sys
positive_words = ["good", "awesome", "excellent", "nice", "amazing", "loved"]
negative_words = ["bad", "poor", "waste"]
for line in sys.stdin:
   if "id" in line:
      continue
   data = line.strip().split(',')
   if len(data) < 8:
      continue
   comment = data[7].lower()
   pos = 0
   neg = 0
   for word in positive_words:
      if word in comment:
         pos += 1
   for word in negative_words:
      if word in comment:
         neg += 1
   if pos > neg:
      print "Positive\t1"
   elif neg > pos:
      print "Negative\t1"
```



```
 GNU nano 2.0.9                          File: mapper.py

#!/usr/bin/env python
import sys

positive_words = ["good", "awesome", "excellent", "nice", "amazing", "loved"]
negative_words = ["bad", "poor", "waste"]

for line in sys.stdin:
    if "id" in line:
        continue

    data = line.strip().split(',')

    if len(data) < 8:
        continue

    comment = data[7].lower()

    pos = 0
    neg = 0

    for word in positive_words:
        if word in comment:
            pos += 1

    for word in negative_words:
        if word in comment:
            neg += 1

    if pos > neg:
        print "Positive\t1"
    elif neg > pos:
        print "Negative\t1"
```

Step-05 : Write a Python reducer script to aggregate mapper output and calculate total counts of each sentiment**.**

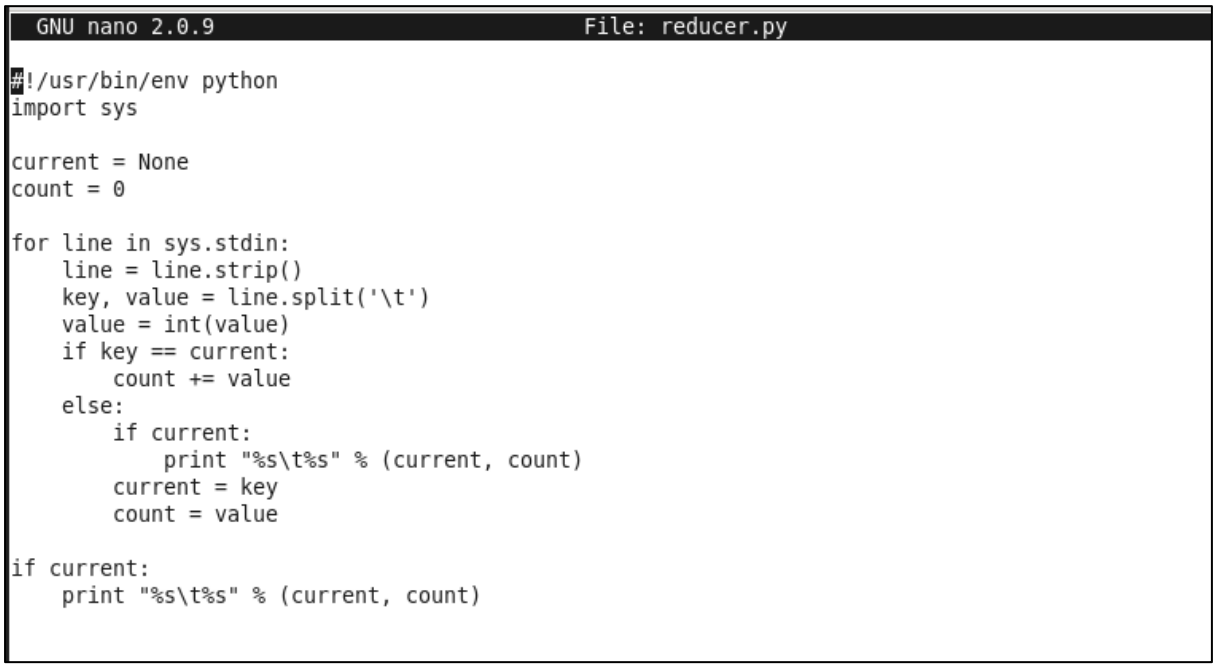
- nano reducer.py

```
#!/usr/bin/env python
import sys

current = None
count = 0

for line in sys.stdin:
   line = line.strip()
   key, value = line.split('\t')
   value = int(value)

   if key == current:
      count += value
   else:
      if current:
         print "%s\t%s" % (current, count)
      current = key
      count = value
if current:
   print "%s\t%s" % (current, count)
```



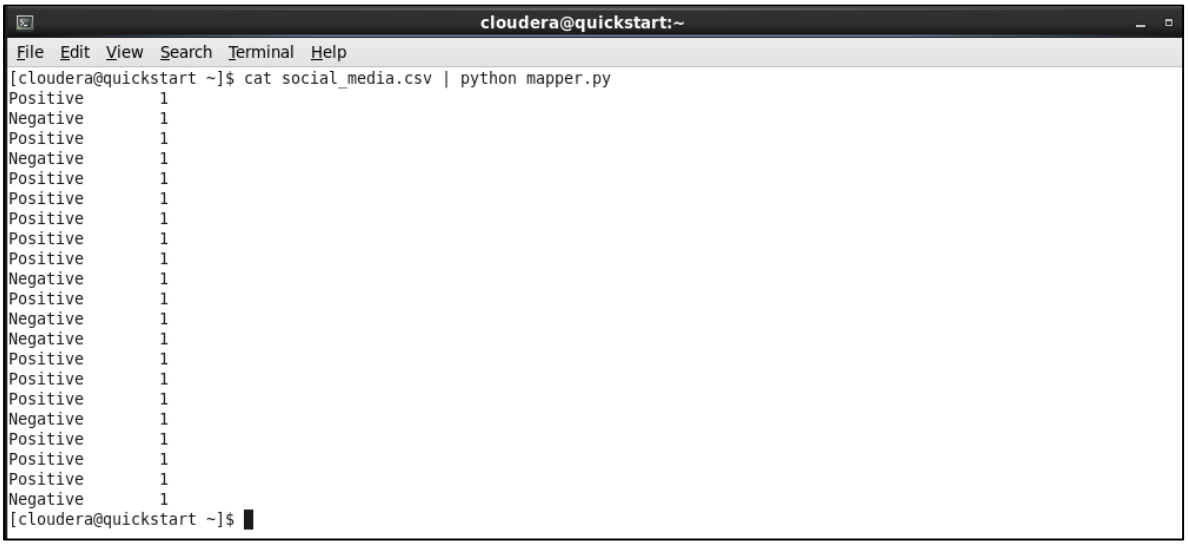
Step-06：Provide execution permissions to mapper and reducer scripts using Linux commands.

- chmod +x mapper.py reducer.py

```
[cloudera@quickstart ~]$ nano mapper.py
[cloudera@quickstart ~]$ nano reducer.py
[cloudera@quickstart ~]$ chmod +x mapper.py reducer.py
```

Step-07 : Test mapper and reducer locally using Linux commands to ensure correctness before running on Hadoop.

- cat social_media.csv | python mapper.py



- cat social_media.csv | python mapper.py | sort | python reducer.py

```
[cloudera@quickstart ~]$ cat social_media.csv | python mapper.py | sort | python reducer.py
Negative        7
Positive        14
```

Step-08 : Run the MapReduce job using Hadoop streaming to process the dataset stored in HDFS.

- hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
  -files mapper.py,reducer.py \
  -input /user/cloudera/social_project/input/social_media.csv \
  -output /user/cloudera/social_project/output \
  -mapper "python mapper.py" \
  -reducer "python reducer.py"

```
[cloudera@quickstart ~]$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
> -files mapper.py,reducer.py \
> -input /user/cloudera/social_project/input/social_media.csv \
> -output /user/cloudera/social_project/output \
> -mapper "python mapper.py" \
> -reducer "python reducer.py"
packageJobJar: [] [/usr/jars/hadoop-streaming-2.6.0-cdh5.4.2.jar] /tmp/streamjob8769527607740608578.jar tmpDir=null
26/04/25 01:03:48 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/25 01:03:48 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/25 01:03:48 INFO mapred.FileInputFormat: Total input paths to process : 1
26/04/25 01:03:48 INFO mapreduce.JobSubmitter: number of splits:2
26/04/25 01:03:49 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1777099304994_0001
26/04/25 01:03:49 INFO impl.YarnClientImpl: Submitted application application_1777099304994_0001
26/04/25 01:03:49 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_17770993049
94_0001/
26/04/25 01:03:49 INFO mapreduce.Job: Running job: job_1777099304994_0001
26/04/25 01:03:59 INFO mapreduce.Job: Job job_1777099304994_0001 running in uber mode : false
26/04/25 01:03:59 INFO mapreduce.Job:  map 0% reduce 0%
26/04/25 01:04:05 INFO mapreduce.Job:  map 50% reduce 0%
26/04/25 01:04:07 INFO mapreduce.Job:  map 100% reduce 0%
26/04/25 01:04:12 INFO mapreduce.Job:  map 100% reduce 100%
26/04/25 01:04:12 INFO mapreduce.Job: Job job_1777099304994_0001 completed successfully
26/04/25 01:04:12 INFO mapreduce.Job: Counters: 49
        File System Counters
                FILE: Number of bytes read=279
                FILE: Number of bytes written=342092
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=2642
                HDFS: Number of bytes written=23
                HDFS: Number of read operations=9
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
        Job Counters
                Launched map tasks=2
```

Step-09 : Display the processed output stored in HDFS to verify the results of MapReduce execution**.**

- hdfs dfs -cat /user/cloudera/social_project/output/part-00000

```
[cloudera@quickstart ~]$ hdfs dfs -cat /user/cloudera/social_project/output/part-00000
Negative        7
Positive        14
[cloudera@quickstart ~]$
```

QUESTION-2:- TO PERFORM DATA ANALYSIS ON THE GIVE SOCIAL MEDIA DATA USING HIVE

❖ AIM:

"TO ANALYZE PROCESSED SOCIAL MEDIA DATA USING HIVE QUERIES."

Step-01 : Start the Hive shell to perform data analysis**.**

- Hive

```
[cloudera@quickstart ~]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive>
```

Step-02 : Create a table in Hive to store processed sentiment data.

- CREATE TABLE sentiment (
  type STRING,
  count INT
)
  ROW FORMAT DELIMITED
  FIELDS TERMINATED BY '\t';

```
hive> CREATE TABLE sentiment (
    > type STRING,
    > count INT
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY '\t';
OK
Time taken: 3.786 seconds
hive>
```

Step-03 : Load the MapReduce output from HDFS into the Hive table**.**

- LOAD DATA INPATH '/USER/CLOUDERA/SOCIAL_PROJECT/OUTPUT' INTO TABLE SENTIMENT;

```
hive> LOAD DATA INPATH '/user/cloudera/social_project/output' INTO TABLE sentiment;
Loading data to table default.sentiment
chgrp: changing ownership of 'hdfs://quickstart.cloudera:8020/user/hive/warehouse/sentiment/part-00000': User does not belong
 to hive
Table default.sentiment stats: [numFiles=1, totalSize=23]
OK
Time taken: 0.564 seconds
hive>
```

Step-04 : Retrieve and display data stored in the Hive table.

- SELECT * FROM SENTIMENT;

```
hive> SELECT * FROM sentiment;
OK
Negative        7
Positive        14
Time taken: 0.506 seconds, Fetched: 2 row(s)
hive>
```

Step-05 : Execute Hive queries such as sorting and filtering to analyze sentiment distribution.

- SELECT * FROM SENTIMENT ORDER BY COUNT DESC;

```
hive> SELECT * FROM sentiment ORDER BY count DESC;
Query ID = cloudera_20260425011010_0dfe2d0c-db9f-436c-bacd-011fb16ee88b
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1777099304994_0002, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1777099304994_0002/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1777099304994_0002
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-25 01:10:38,950 Stage-1 map = 0%,  reduce = 0%
2026-04-25 01:10:49,902 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 1.12 sec
2026-04-25 01:10:56,126 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 2.21 sec
MapReduce Total cumulative CPU time: 2 seconds 210 msec
Ended Job = job_1777099304994_0002
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 2.21 sec   HDFS Read: 5571 HDFS Write: 23 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 210 msec
OK
Positive        14
Negative        7
Time taken: 25.742 seconds, Fetched: 2 row(s)
hive>
```

OUTPUT:

- **HIVE SUCCESSFULLY DISPLAYED:**
  - SENTIMENT CLASSIFICATION RESULTS (POSITIVE AND NEGATIVE COUNTS)
  - SORTED SENTIMENT DATA BASED ON FREQUENCY